

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ОТЧЕТ

по Лабораторной работе № 6

«Работа с БД в СУБД MongoDB»

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Чан Тхи Лиен

Факультет прикладной информатики

Группа К3240

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии 2023

Преподаватель Говорова Марина Михайловна

Белов Александр Олегович

Санкт-Петербург

2025/2026

1. Наименование работы.

Работа с БД в СУБД MongoDB

2. Цель работы.

Овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

3. Выполнение

Практическое задание 2.1.1:

1) Создайте базу данных *learn*

```
test> use learn
switched to db learn
learn>
```

2) Заполните коллекцию единорогов *unicorns*

```
learn> db.unicorns.insertMany([
  {name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63},
  {name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43},
  {name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182},
  {name: 'Rooooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99},
  {name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80},
  {name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40},
  {name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39},
  {name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2},
  {name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33},
  {name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54},
  {name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'}
]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a186ed2430a40b1dfabc114'),
    '1': ObjectId('6a186ed2430a40b1dfabc115'),
    '2': ObjectId('6a186ed2430a40b1dfabc116'),
    '3': ObjectId('6a186ed2430a40b1dfabc117'),
    '4': ObjectId('6a186ed2430a40b1dfabc118'),
    '5': ObjectId('6a186ed2430a40b1dfabc119'),
    '6': ObjectId('6a186ed2430a40b1dfabc11a'),
    '7': ObjectId('6a186ed2430a40b1dfabc11b'),
    '8': ObjectId('6a186ed2430a40b1dfabc11c'),
    '9': ObjectId('6a186ed2430a40b1dfabc11d'),
    '10': ObjectId('6a186ed2430a40b1dfabc11e')
  }
}
```

3) Используя второй способ, вставьте в коллекцию единорогов документ

```
learn> var document = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165};
| db.unicorns.insertOne(document);
{
  acknowledged: true,
  insertedId: ObjectId('6a186efd430a40b1dfabc11f')
}
learn>
```

4) Проверьте содержимое коллекции с помощью метода *find*

```
]learn> db.unicorns.find();
[
  {
    _id: ObjectId('6a186ed2430a40b1dfabc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a186ed2430a40b1dfabc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a186ed2430a40b1dfabc116'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a186ed2430a40b1dfabc117'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a186ed2430a40b1dfabc118'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a186ed2430a40b1dfabc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a186ed2430a40b1dfabc11a'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
```

Практическое задание 2.2.1:

- 1) Сформируйте запросы для вывода списков самцов и самок единорогов. Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени.

```
learn> db.unicorns.find({gender: 'm'}).sort({name: 1});
| db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3);
[
  {
    _id: ObjectId('6a186ed2430a40b1dfabc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a186ed2430a40b1dfabc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a186ed2430a40b1dfabc11c'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
learn>
```

- 2) Найдите всех самок, которые любят carrot. Ограничьте этот список первой особью с помощью функций `findOne` и `limit`.

```
learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'});
{
  _id: ObjectId('6a186ed2430a40b1dfabc115'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn> _
```

```

learn> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
[
  {
    _id: ObjectId('6a186ed2430a40b1dfabc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn> _

```

Практическое задание 2.2.2:

Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле.

```

learn> db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0});
[
  {
    _id: ObjectId('6a186ed2430a40b1dfabc114'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a186ed2430a40b1dfabc116'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  },
  {
    _id: ObjectId('6a186ed2430a40b1dfabc117'),
    name: 'Rooooooodles',
    weight: 575,
    vampires: 99
  },
  {
    _id: ObjectId('6a186ed2430a40b1dfabc11a'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('6a186ed2430a40b1dfabc11b'),
    name: 'Raleigh',
    weight: 421,
    vampires: 2
  },
  {
    _id: ObjectId('6a186ed2430a40b1dfabc11d'),
    name: 'Pilot',
    weight: 650,
    vampires: 54
  },
  {
    _id: ObjectId('6a186efd430a40b1dfabc11f'),
    name: 'Dunx',
    weight: 704,
    vampires: 165
  }
]
learn>

```

Практическое задание 2.2.3:

Вывести список единорогов в обратном порядке добавления.

```
learn> db.unicorns.find().sort({$natural: -1});
[
  {
    _id: ObjectId('6a186efd430a40b1dfabc11f'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a186ed2430a40b1dfabc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a186ed2430a40b1dfabc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a186ed2430a40b1dfabc11c'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a186ed2430a40b1dfabc11b'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a186ed2430a40b1dfabc11a'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a186ed2430a40b1dfabc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
]
```

Практическое задание 2.1.4:

Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

```
]
learn> db.unicorns.find({}, {name: 1, loves: {$slice: 1}, _id: 0});
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]
learn>
```

Практическое задание 2.3.1:

Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

```
]
learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0});
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn>
```

Практическое задание 2.3.2:

Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

```
learn> db.unicorns.find({gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}}, {_id: 0});
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
learn>
```

Практическое задание 2.3.3:

Найти всех единорогов, не имеющих ключ vampires.

```
learn> db.unicorns.find({ vampires: { $exists: false } })
[
  {
    _id: ObjectId('6a186ed2430a40b1dfabc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> _
```

Практическое задание 2.3.4:

Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

```
learn> db.unicorns.find({ gender: 'm' }, { name: 1, loves: { $slice: 1 }, _id: 0 }).sort({ name: 1 })
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
learn> _
```

Практическое задание 3.1.1:

1) *Создайте коллекцию towns, включающую следующие документы:*


```
learn> db.towns.insertMany([
  {
    name: "Punxsutawney",
    population: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: ["phil the groundhog"],
    mayor: { name: "Jim Wehrle" }
  },
  {
    name: "New York",
    population: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["status of liberty", "food"],
    mayor: { name: "Michael Bloomberg", party: "I" }
  },
  {
    name: "Portland",
    population: 528000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: { name: "Sam Adams", party: "D" }
  }
]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a187434430a40b1dfabc120'),
    '1': ObjectId('6a187434430a40b1dfabc121'),
    '2': ObjectId('6a187434430a40b1dfabc122')
  }
}
learn>
```

- 2) Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I"). Вывести только название города и информацию о мэре.

```
learn> db.towns.find({ "mayor.party": "I" }, { name: 1, mayor: 1, _id: 0 })
[
  { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
]
learn>
```

- 3) Сформировать запрос, который возвращает список беспартийных мэров (party отсутствует). Вывести только название города и информацию о мэре.

```
learn> db.towns.find({ "mayor.party": { $exists: false } }, { name: 1, mayor: 1, _id: 0 })
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
learn>
```

Практическое задание 3.1.2:

- 1) Содержание коллекции единорогов unicorns:

```
learn> db.unicorns.insertMany([
  {name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63},
  {name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43},
  {name: 'Unicrom', loves: ['enengon', 'redbull'], weight: 984, gender: 'm', vampires: 182},
  {name: 'Rooooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99},
  {name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80},
  {name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40},
  {name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39},
  {name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2},
  {name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33},
  {name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54},
  {name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'},
  {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a1875eb430a40b1dfabc123'),
    '1': ObjectId('6a1875eb430a40b1dfabc124'),
    '2': ObjectId('6a1875eb430a40b1dfabc125'),
    '3': ObjectId('6a1875eb430a40b1dfabc126'),
    '4': ObjectId('6a1875eb430a40b1dfabc127'),
    '5': ObjectId('6a1875eb430a40b1dfabc128'),
    '6': ObjectId('6a1875eb430a40b1dfabc129'),
    '7': ObjectId('6a1875eb430a40b1dfabc12a'),
    '8': ObjectId('6a1875eb430a40b1dfabc12b'),
    '9': ObjectId('6a1875eb430a40b1dfabc12c'),
    '10': ObjectId('6a1875eb430a40b1dfabc12d'),
    '11': ObjectId('6a1875eb430a40b1dfabc12e')
  }
}
learn>
```

2) Сформировать функцию для вывода списка самцов единорогов.

```
learn> db.towns.find({ "mayor.party": { $exists: false } }, { name: 1, mayor: 1, _id: 0 })
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
learn> function getMaleUnicorns() {
  |   return db.unicorns.find({ gender: 'm' });
  | }
[Function: getMaleUnicorns]
```

3) Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.

```
learn> var cursor = getMaleUnicorns().sort({ name: 1 }).limit(2); null;
null
```

4) Вывести результат, используя `forEach`.

```
learn> cursor.forEach(function(obj) {
  |   print(obj.name);
  | });
Dunx
Horny
```

Практическое задание 3.2.1:

Вывести количество самок единорогов весом от полутонны до 600 кг.

```
learn> db.unicorns.find({ gender: 'f', weight: { $gte: 500, $lte: 600 } }).count()
2
```

Практическое задание 3.2.2:

Вывести список предпочтений.

```
learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
```

Практическое задание 3.2.3:

Посчитать количество особей единорогов обоих полов.

```
learn> db.unicorns.aggregate([
  {
    $group: {
      _id: "$gender",
      count: { $sum: 1 }
    }
  }
])
[ { _id: 'm', count: 7 }, { _id: 'f', count: 5 } ]
```

Практическое задание 3.3.1:

1) Выполнить команду:

```
learn> db.unicorns.insertOne({ name: 'Barney', loves: ['grape'], weight: 340, gender: 'm' })
{
  acknowledged: true,
  insertedId: ObjectId('6a1878b3430a40b1dfabc13c')
}
```

2) Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.find({ name: 'Barney' })
[
  {
    _id: ObjectId('6a1878b3430a40b1dfabc13c'),
    name: 'Barney',
    loves: [ 'grape' ],
    weight: 340,
    gender: 'm'
  }
]
```

Практическое задание 3.3.2:

1) Для самки единорога Ауна внести изменения в БД: теперь ее вес 800, она убила 51 вампира.

```

MongoServerError: replaceOne() update document requires atomic operators
learn> db.unicorns.replaceOne({ name: 'Ayna' }, { name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 800, gender: 'f', vampires: 51 })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}

```

2) Проверить содержимое коллекции unicorns.

```

learn> db.unicorns.find({ name: 'Ayna' })
[
  {
    _id: ObjectId('6a1877a7430a40b1dfabc135'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  }
]
learn>

```

Практическое задание 3.3.3:

1) Для самца единорога Raleigh внести изменения в БД: теперь он любит рэдбул.

```

learn> db.unicorns.updateOne(
  { name: 'Raleigh' },
  { $set: { loves: ['apple', 'sugar', 'redbull'] } }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}

```

2) Проверить содержимое коллекции unicorns.

```

learn> db.unicorns.find({ name: 'Raleigh' })
[
  {
    _id: ObjectId('6a1877a7430a40b1dfabc137'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar', 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]

```

Практическое задание 3.3.4:

1) Всем самцам единорогов увеличить количество убитых вампиров на 5.

```
learn> db.unicorns.updateMany(
  { gender: 'm' },
  { $inc: { vampires: 5 } }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 8,
  modifiedCount: 8,
  upsertedCount: 0
}
```

2) Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.find({ gender: 'm' }, { name: 1, vampires: 1, _id: 0 })
[
  { name: 'Horny', vampires: 68 },
  { name: 'Unicrom', vampires: 187 },
  { name: 'Rooooooodles', vampires: 104 },
  { name: 'Kenny', vampires: 44 },
  { name: 'Raleigh', vampires: 7 },
  { name: 'Pilot', vampires: 59 },
  { name: 'Dunx', vampires: 170 },
  { name: 'Barny', vampires: 5 }
]
```

Практическое задание 3.3.5:

1) Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.

```
learn> db.towns.updateOne(
  { name: "Portland" },
  { $unset: { "mayor.party": 1 } }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

2) Проверить содержимое коллекции towns.

```
learn> db.towns.find({ name: "Portland" })
[
  {
    _id: ObjectId('6a187434430a40b1dfabc122'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
```

Практическое задание 3.3.6:

- 1) Изменить информацию о самце единорога Pilot: теперь он любит и шоколад.

```
learn> db.unicorns.updateOne(
|   { name: 'Pilot' },
|   { $push: { loves: 'chocolate' } }
| )
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 1,
|   modifiedCount: 1,
|   upsertedCount: 0
| }
```

- 2) Проверить содержимое коллекции unicorns.

```
learn> db.unicorns.find({ name: 'Pilot' })
[
|   {
|     _id: ObjectId('6a1877a7430a40b1dfabc139'),
|     name: 'Pilot',
|     loves: [ 'apple', 'watermelon', 'chocolate' ],
|     weight: 650,
|     gender: 'm',
|     vampires: 59
|   }
| ]
```

Практическое задание 3.3.7:

- 1) Изменить информацию о самке единорога Aurora: теперь она любит еще и сахар, и лимоны.

```
learn> db.unicorns.updateOne(
|   { name: 'Aurora' },
|   { $push: { loves: { $each: ['sugar', 'lemon'] } } }
| )
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 1,
|   modifiedCount: 1,
|   upsertedCount: 0
| }
```

- 2) Проверить содержимое коллекции unicorns.

```

learn> db.unicorns.find({ name: 'Aurora' })
[
  {
    _id: ObjectId('6a1877a7430a40b1dfabc131'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]

```

Практическое задание 3.4.1:

- 1) Создайте коллекцию `towns`, включающую следующие документы:

```

learn> db.towns.drop();
db.towns.insertMany([
  {
    name: "Punxsutawney",
    popujatiuon: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: ["phil the groundhog"],
    mayor: { name: "Jim Wehrle" }
  },
  {
    name: "New York",
    popujatiuon: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["status of liberty", "food"],
    mayor: { name: "Michael Bloomberg", party: "I" }
  },
  {
    name: "Portland",
    popujatiuon: 528000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: { name: "Sam Adams", party: "D" }
  }
]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a187b7e430a40b1dfabc13d'),
    '1': ObjectId('6a187b7e430a40b1dfabc13e'),
    '2': ObjectId('6a187b7e430a40b1dfabc13f')
  }
}
learn>

```

- 2) Удалите документы с беспартийными мэрами.

```

learn> db.towns.deleteMany({ "mayor.party": { $exists: false } })
{ acknowledged: true, deletedCount: 1 }
learn>

```

- 3) Проверьте содержание коллекции.

```
learn> db.towns.find()
[
  {
    _id: ObjectId('6a187b7e430a40b1dfabc13e'),
    name: 'New York',
    population: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a187b7e430a40b1dfabc13f'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
```

4) Очистите коллекцию.

```
learn> db.towns.drop()
true
```

5) Просмотрите список доступных коллекций.

```
learn> show collections
unicorns
```

Практическое задание 4.1.1:

1) Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.

```
learn> db.zones.insertMany([
  {
    _id: "FOREST",
    fullName: "Enchanted Forest",
    description: "Khu rừng phép thuật ngập tràn ánh sáng mặt trời và cây xanh."
  },
  {
    _id: "MOUNTAIN",
    fullName: "Misty Mountains",
    description: "Vùng núi sương mù hiếm trở, nơi có không khí lạnh giá."
  }
]);
{ acknowledged: true, insertedIds: { '0': 'FOREST', '1': 'MOUNTAIN' } }
```

2) Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.


```

learn> db.unicorns.updateOne(
  { name: 'Horny' },
  { $set: { zone_id: "FOREST" } }
);
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
learn> db.unicorns.updateOne(
  { name: 'Aurora' },
  { $set: { zone_id: "FOREST" } }
);
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
learn> db.unicorns.updateOne(
  { name: 'Unicrom' },
  { $set: { zone_id: "MOUNTAIN" } }
);
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}

```

3) Проверьте содержание коллекции единорогов.

```

learn> db.unicorns.find({ zone_id: { $exists: true } })
[
  {
    _id: ObjectId('6a187cfc430a40b1dfabc140'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63,
    zone_id: 'FOREST'
  },
  {
    _id: ObjectId('6a187cfc430a40b1dfabc141'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43,
    zone_id: 'FOREST'
  },
  {
    _id: ObjectId('6a187cfc430a40b1dfabc142'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182,
    zone_id: 'MOUNTAIN'
  }
]

```

Практическое задание 4.2.1:

- 1) Проверьте, можно ли задать для коллекции *unicorns* индекс для ключа *name* с флагом *unique*.

```
learn> db.unicorns.createIndex({ name: 1 }, { unique: true })
name_1
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_', },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
```

Практическое задание 4.3.1:

- 1) Получите информацию о всех индексах коллекции *unicorns*.

```
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_', },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
```

- 2) Удалите все индексы, кроме индекса для идентификатора.

```
learn> db.unicorns.dropIndex("name_1")
{ nIndexesWas: 2, ok: 1 }
learn>
```

- 3) Попробуйте удалить индекс для идентификатора.

```
learn> db.unicorns.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index
learn>
```

Практическое задание 4.4.1:

- 1) Создайте объемную коллекцию *numbers*, задействовав курсор:

```
learn> var bulkData = [];
| for (let i = 0; i < 100000; i++) {
|   bulkData.push({ value: i });
| }
| db.numbers.insertMany(bulkData); null;
null
```

- 2) Выберите последних четыре документа.

```

learn> db.numbers.find().sort({ value: -1 }).limit(4).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'AE5753F2',
    planCacheShapeHash: 'AE5753F2',
    planCacheKey: 'AE69D0D4',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'SORT',
      sortPattern: { value: -1 },
      memLimit: 104857600,
      limitAmount: 4,
      type: 'simple',
      inputStage: { stage: 'COLLSCAN', nss: 'learn.numbers', direction: 'forward' }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 96,
    totalKeysExamined: 0,
    totalDocsExamined: 100000,
    executionStages: {
      isCached: false,
      stage: 'SORT',
      nReturned: 4,
      executionTimeMillisEstimate: 84,
      works: 100006,
      advanced: 4,
      needTime: 100001,
      needYield: 0,
      saveState: 5,
      restoreState: 5,
      isEOF: 1,
      sortPattern: { value: -1 },
      memLimit: 104857600,
      limitAmount: 4,
      type: 'simple',
      totalDataSizeSorted: 260,
      usedDisk: false,

```

```

    type: 'simple',
    totalDataSizeSorted: 260,
    usedDisk: false,
    spills: 0,
    spilledRecords: 0,
    spilledBytes: 0,
    spilledDataStorageSize: 0,
    peakTrackedMemBytes: 260,
    inputStage: {
      stage: 'COLLSCAN',
      nReturned: 100000,
      executionTimeMillisEstimate: 42,
      works: 100001,
      advanced: 100000,
      needTime: 0,
      needYield: 0,
      saveState: 5,
      restoreState: 5,
      isEOF: 1,
      nss: 'learn.numbers',
      direction: 'forward',
      docsExamined: 100000
    }
  },
  queryShapeHash: 'A1C8CFCE9F916AB3A6ABCC8ABBB22506390F4D987582D3F926F0F2B36CA78396',
  command: {
    find: 'numbers',
    filter: {},
    sort: { value: -1 },
    limit: 4,
    '$db': 'learn'
  },
  serverInfo: {
    host: 'TRANLIEN',
    port: 27017,
    version: '8.3.2',
    gitVersion: '89f54eb32e217d2f7bfcad50e5919f3033bb9611'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}

```

- 3) Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра `executionTimeMillis`)
- : Отобразится "COLLSCAN" (сканирование коллекции) в сочетании с "SORT". Это означает, что MongoDB должна прочитать все 100 000 строк в память, прежде чем выполнить сортировку для поиска последних 4 строк.
 - `executionTimeMillis`: Время выполнения
 - `totalDocsExamined`: 100000
- 4) Создайте индекс для ключа `value`.

```
learn> db.numbers.createIndex({ value: 1 })
value_1
```

5) Получите информацию о всех индексах коллекции *numbers*.

```
learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
```

6) Выполните запрос 2.

```
learn> db.numbers.find().sort({ value: -1 }).limit(4).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'AE5753F2',
    planCacheShapeHash: 'AE5753F2',
    planCacheKey: 'AE69D0D4',
    optimizationTimeMillis: 1,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 4,
      inputStage: {
        stage: 'FETCH',
        nss: 'learn.numbers',
        inputStage: {
          stage: 'IXSCAN',
          nss: 'learn.numbers',
          keyPattern: { value: 1 },
          indexName: 'value_1',
          isMultiKey: false,
          multiKeyPaths: { value: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'backward',
          indexBounds: { value: [ '[MaxKey, MinKey]' ] }
        }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 1,
    totalKeysExamined: 4,
    totalDocsExamined: 4,
    executionStages: {
      isCached: false,
      stage: 'LIMIT',
      nReturned: 4,
      executionTimeMillisEstimate: 0,
      works: 5,
      advanced: 4,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
    }
  }
}
```

```

    restoreState: 0,
    isEOF: 0,
    nss: 'learn.numbers',
    docsExamined: 4,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 4,
      executionTimeMillisEstimate: 0,
      works: 4,
      advanced: 4,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 0,
      nss: 'learn.numbers',
      keyPattern: { value: 1 },
      indexName: 'value_1',
      isMultiKey: false,
      multiKeyPaths: { value: [] },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'backward',
      indexBounds: { value: [ '[MaxKey, MinKey]' ] },
      keysExamined: 4,
      seeks: 1,
      dupsTested: 0,
      dupsDropped: 0,
      peakTrackedMemBytes: 0
    }
  }
},
queryShapeHash: 'A1C8CFCE9F916AB3A6ABCC8ABBB22506390F4D987582D3F926F0F2B36CA78396',
command: {
  find: 'numbers',
  filter: {},
  sort: { value: -1 },
  limit: 4,
  '$db': 'learn'
},
serverInfo: {
  host: 'TRANLIEN',
  port: 27017,
  version: '8.3.2',
  gitVersion: '89f54eb32e217d2f7bfcad50e5919f3033bb9611'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,

```

7) Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?

- stage: 'IXSCAN'
- executionTimeMillisEstimate: 0
- totalDocsExamined: 4

8) Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?

- Сравнение времени выполнения: * Без индекса: `executionTimeMillis` составляет приблизительно ~50 мс (Сканирование всей таблицы - COLLSCAN).
- С индексом: `executionTimeMillis` снижается до 0 мс (Сканирование по индексному дереву - IXSCAN).
- Запросы после индексирования (с индексом) значительно эффективнее и оптимизированы. Индекс действует как оглавление, позволяя системе мгновенно находить элементы в конце списка, не расходуя ресурсы ЦП и ОЗУ на обход огромного массива данных всей коллекции.

4. Вывод

В ходе выполнения лабораторной работы были успешно освоены практические навыки работы с документоориентированной СУБД MongoDB с использованием интерфейса командной строки MongoDB Shell (**mongosh**).

В процессе выполнения практических заданий были изучены и закреплены следующие ключевые аспекты работы с NoSQL-базами данных:

- Манипуляция данными и CRUD-операции: Были освоены методы эффективной вставки документов (с использованием **insertMany** для предотвращения дублирования и синтаксических ошибок), выборки данных (**find**, **findOne**) с применением условных операторов фильтрации (**gte**, **lte**, **all**, **exists**).
- Использование проекции, сортировки и курсоров: На практике отработаны механизмы ограничения объема выводимых данных (**limit**), сортировки результатов в лексикографическом порядке (**sort**), а также управление курсорами (**Cursor**) в связке с циклами JavaScript (**forEach**) для построчной обработки документов.
- Обновление и структурирование документов: Изучены различия между полным замещением документа с помощью **replaceOne** и точечным изменением полей с помощью атомарных операторов. Были успешно применены операторы **set** (изменение значения), **inc** (инкремент числовых данных), **unset** (удаление ключа) и **push/each** (модификация массивов внутри документов).
- Связывание данных (Peaccessing): Отработан метод автоматического связывания документов из разных коллекций (на примере **unicorns** и **zones**) путем явного внедрения идентификатора **_id** целевой коллекции в качестве ссылочного поля.
- Индексация и оптимизация производительности: Был успешно создан и протестирован уникальный индекс (**unique: true**), обеспечивающий целостность данных на уровне СУБД.
 - Доказана невозможность удаления системного индекса первичного ключа (**_id**), что подтверждает встроенные механизмы защиты целостности MongoDB.
 - На примере объемной коллекции из 100.000 документов с помощью инструмента анализа `.explain("executionStats")` было проведено сравнение эффективности запросов. На практике доказано, что переход от полного сканирования коллекции (COLLSCAN) к сканированию по индексу (IXSCAN) снижает время выполнения запроса (**executionTimeMillis**) с десятков миллисекунд до 0 мс, сводя к минимуму нагрузку на процессор и оперативную память.

